KCPC Week 4 Problems
Introduction to Graphs: Traversals, Structure, and Modelling

Welcome to Workshop 4! This problem sheet accompanies the *Introduction to Graphs* slides and is organised into three progressive sections:

- **Section 1: Warm-Up Exercises:** Breadth-First Search (BFS) and Depth-First Search (DFS) for connected components, unweighted shortest paths, cycle detection, and 2-colouring.
- **Section 2: Core Practice Exercises:** Functional graphs, degree characterisations of cycles, dynamic programming on Directed Acyclic Graphs (DAGs), and tree diameter.
- **Section 3: Extended Homework:** Bipartite graph construction and reverse sink-tree analysis on functional graphs.

1 Warm-Up Exercises (Foundational Traversals)

1.1 Building Roads

Link: https://cses.fi/problemset/task/1666
Prerequisites: Adjacency list representation, connected components via DFS or BFS.

Byteland has n cities ($1 \leq n \leq 10^5$) and m bidirectional roads ($1 \leq m \leq 2 \cdot 10^5$) connecting pairs of cities. The goal is to construct new roads so that there is a route between any two cities.

Task: Determine the minimum number of new roads required to make the entire road network connected, and output a valid list of roads (u_i, v_i) to build.

Hint: Finding Connected Components

- Loop through vertices $i = 1, \ldots, n$.
- Whenever i is unvisited, start DFS or BFS from i. This traversal visits exactly the vertices in i's connected component.
- Record i as that component's representative and continue scanning.

Application: Building Roads

If the graph has C components represented by $r_1, \ldots, r_C$, add roads

$$(r_1, r_2), (r_2, r_3), \ldots, (r_{C-1}, r_C).$$

Each road joins two components, so $C - 1$ roads are sufficient and necessary.

Complexity: $O(N + E)$ time with an adjacency list.

1.2 Message Route

Link: https://cses.fi/problemset/task/1667
Prerequisites: Breadth-First Search (BFS), shortest unweighted paths, parent pointer reconstruction.

Syrjälä's network has n computers ($2 \leq n \leq 10^5$) and m undirected connections ($1 \leq m \leq 2 \cdot 10^5$). Uolevi wants to send a message from computer 1 to computer n.

Task: Decide whether computer n is reachable from computer 1. If so, find the minimum number of computers along a valid route (including both endpoints 1 and n) and print the sequence of computers on one such shortest route; otherwise, print `IMPOSSIBLE`.

1.3 Round Trip

Link: https://cses.fi/problemset/task/1669
Prerequisites: Depth-First Search (DFS), back-edges in undirected graphs, cycle reconstruction.

Byteland has n cities ($1 \leq n \leq 10^5$) and m bidirectional roads ($1 \leq m \leq 2 \cdot 10^5$). You want to design a round trip that begins in some city, passes through at least two other distinct cities, visits no intermediate city more than once, and returns to the starting city.

Task: Find any simple cycle of length at least 3 in the undirected graph and print the number of vertices on the route (counting the start/end city twice, so $k \geq 4$) followed by the sequence of cities visited. If the graph is a forest (acyclic), print `IMPOSSIBLE`.

Hint: Finding Cycles in Undirected Graphs

During DFS, track each vertex's parent.

1. If an edge from v discovers an unvisited neighbour u, set $\texttt{parent}[u] = v$ and continue DFS.
2. Ignore the edge back to $\texttt{parent}[v]$; it is just the same undirected edge in reverse.
3. If u is already visited and $u \neq \texttt{parent}[v]$, then in undirected DFS u is an ancestor of v, and the edge (v, u) closes a cycle with the tree path from u to v.

Reconstruct the cycle by following parent pointers from v back to u.

Traversal and reconstruction take $O(N + E)$ time.

1.4 Building Teams

Link: https://cses.fi/problemset/task/1668
Prerequisites: Bipartite graphs, odd-cycle characterisation, 2-colouring via BFS or DFS.

There are n pupils ($1 \leq n \leq 10^5$) in Uolevi's class and m friendships ($1 \leq m \leq 2 \cdot 10^5$) between them. Your task is to divide the pupils into two teams (labelled 1 and 2) such that no two friends belong to the same team.

Task: Output a valid team assignment (1 or 2) for each pupil $1, 2, \ldots, n$, or print `IMPOSSIBLE` if the friendship graph is not bipartite.

Definition (*Building Teams*): A graph is **bipartite** if its vertices can be split into two teams so every edge connects opposite teams.

Theorem: A graph is bipartite if and only if it has no odd-length cycle.

- Even cycles alternate colours consistently.
- An odd cycle eventually requires one edge to connect vertices of the same colour.

Hint: 2-Colouring with BFS or DFS

1. For each uncoloured vertex s, set $\texttt{col}[s] = 1$ and start BFS/DFS.
2. For each edge (v, u):
   - If $\texttt{col}[u] = 0$, assign the opposite colour: $\texttt{col}[u] = 3 - \texttt{col}[v]$.
   - If $\texttt{col}[u] = \texttt{col}[v]$, the graph is not bipartite; output `IMPOSSIBLE`.
3. Repeat from every uncoloured vertex so disconnected components are also checked.

Complexity: $O(N + E)$ time and $O(N + E)$ space with an adjacency list.

2 Core Practice Exercises (Structure, Trees, and DAGs)

2.1 Badge

Link: https://codeforces.com/contest/1020/problem/B
Prerequisites: Directed graphs, functional graphs (out-degree 1), cycle entry detection.

In a school of n students ($1 \leq n \leq 1000$), whenever the teacher catches student a playing a prank, a hole is punched in a's badge and student a blames student p_a ($1 \leq p_a \leq n$). The teacher then moves to p_a, punches a hole in p_a's badge, and continues following the chain $x \to p_x$ until reaching a student whose badge already has a hole (who then receives a second hole).

Task: For every possible starting student $a \in \{1, 2, \ldots, n\}$, determine which student is the first to receive two holes in their badge.

Hint: Functional Graphs

A functional graph is a directed graph where every vertex has exactly one outgoing edge, $u \to f(u)$.

- Following edges from any starting vertex eventually reaches a directed cycle.
- Each component consists of one directed cycle with trees feeding into its cycle vertices (a ρ-shape).
- **Badge:** Repeatedly follow the successor until a vertex is visited twice; that repeated vertex is the first student receiving a second hole.

2.2 Cyclic Components

Link: https://codeforces.com/problemset/problem/977/E
Prerequisites: Connected components, vertex degree invariants.

You are given an undirected simple graph with n vertices ($1 \leq n \leq 2 \cdot 10^5$) and m edges ($0 \leq m \leq 2 \cdot 10^5$). A connected component is a *simple cycle* if its vertices can be ordered $v_1, v_2, \ldots, v_k$ ($k \geq 3$) such that the edges in the component are precisely $(v_1, v_2), (v_2, v_3), \ldots, (v_{k-1}, v_k), (v_k, v_1)$ and no others.

Task: Count how many connected components of the graph are simple cycles.

Hint: Recognising Cyclic Components

A connected component of a simple undirected graph is a simple cycle if and only if every vertex in it has degree 2.

- A cycle gives every vertex exactly two incident edges.
- Conversely, in a finite connected component where every vertex has degree 2, following edges must eventually return to the start. Connectivity rules out vertices outside that cycle.

Hint: Applying the Degree Test

The goal is to count connected components that are simple cycles.

1. For each unvisited vertex, traverse its entire component.
2. Check whether every vertex in the component has degree 2.
3. Count the component if and only if that check succeeds.

The traversal ensures the component is connected; the degree test then characterises a cycle.

Complexity: $O(N + E)$ for the whole graph.

2.3 DAG Number of Routes (Game Routes)

Link: https://cses.fi/problemset/task/1681
Prerequisites: Directed Acyclic Graphs (DAGs), topological sorting, dynamic programming on DAGs.

A game has n levels ($1 \leq n \leq 10^5$) connected by m directed teleporters ($1 \leq m \leq 2 \cdot 10^5$). The teleporter network contains no directed cycles (it is a DAG).

Task: Calculate the total number of distinct routes from level 1 to level n, modulo 10^9+7.

Hint: Topological Sorting a DAG

A **Directed Acyclic Graph (DAG)** has no directed cycles. Its vertices can be ordered so every edge $u \to v$ goes forward in the order.

Kahn's Algorithm (BFS):

1. Compute the in-degree `indeg`$[v]$ of every vertex.

2. Add every vertex with in-degree 0 to a queue.
3. Pop u, append it to the order, and decrement the in-degree of each outgoing neighbour v.
4. Whenever `indeg`$[v]$ reaches 0, add v to the queue.

Each vertex and edge is processed once, so the algorithm takes $O(N + E)$ time. If fewer than N vertices are output, the graph contains a directed cycle.

A topological order ensures that when a vertex is processed, all contributions from its predecessors have already been calculated.

Counting paths (*DAG Number of Routes*): Set dp[1] = 1. For each edge $u \to v$ in topological order, add dp$[u]$ to dp$[v]$ modulo $10^9 + 7$.

$$\text{dp}[v] = \sum_{u \to v} \text{dp}[u] \pmod{10^9 + 7}.$$

2.4 Longest Path

Link: https://atcoder.jp/contests/dp/tasks/dp_g
Prerequisites: Directed Acyclic Graphs (DAGs), memoized DFS or topological order DP.

You are given a simple Directed Acyclic Graph (DAG) G with N vertices ($2 \le N \le 10^5$) and M directed edges $x_i \to y_i$ ($1 \le M \le 10^5$).

Task: Find the length (number of edges) of the longest directed path in G.

Longest path (*AtCoder DP G*): Process vertices in reverse topological order and use

$$\text{dp}[u] = \max_{u \to v}(1 + \text{dp}[v]),$$

with dp$[u] = 0$ for a sink. The longest path in the graph is $\max_u$ dp$[u]$. Both algorithms run in $O(N + E)$ time.

2.5 Tree Diameter

Link: https://cses.fi/problemset/task/1131
Prerequisites: Tree properties (unique paths), Two-BFS / Two-DFS farthest-node theorem.
Note: Try solving it without dynamic programming.

You are given an undirected tree consisting of n nodes ($1 \le n \le 2 \cdot 10^5$) and $n - 1$ edges. The *diameter* of a tree is the maximum distance (number of edges) between any two nodes.

Task: Compute the diameter of the tree in $O(n)$ time using graph traversals alone (without subtree dynamic programming).

Hint: Tree Diameter in Two Traversals

1. Pick an arbitrary vertex r and run BFS/DFS to find a farthest vertex u.

2. Run a second BFS/DFS from u to find a farthest vertex v.
3. The distance $\text{dist}(u, v)$ is the tree diameter.

Why it works: Paths in a tree are unique. The farthest vertex from an arbitrary root is an endpoint of a longest path; the second traversal reaches the other endpoint.

Complexity: Two linear traversals take $O(N)$ time and $O(N)$ space.

3 Extended Homework (Advanced Graph Modelling)

3.1 Rudolf and Subway

Link: https://codeforces.com/contest/1941/problem/G
Prerequisites: Auxiliary bipartite graph construction (star expansion), unweighted BFS.

A subway network has n stations and m bidirectional tracks ($\sum n, \sum m \leq 2 \cdot 10^5$). Each track (u_i, v_i) belongs to a subway line of colour c_i ($1 \leq c_i \leq 2 \cdot 10^5$). Riding along any sequence of connected tracks of the *same* colour costs 1 ticket, and switching to a track of a different colour costs an additional ticket.

Task: Given a starting station b and destination station e, find the minimum number of subway lines Rudolf must use to travel from b to e (0 if $b = e$).

Hint: Auxiliary Graph Modelling

Riding tracks of the same colour costs one ticket. Joining every pair of stations on one colour directly can create $O(N^2)$ edges.

Star expansion:

- Create an auxiliary vertex L_c for each colour c.
- Replace every track (u, v, c) by edges $u - L_c$ and $v - L_c$.
- Riding one line from station u to station v corresponds to $u \to L_c \to v$.

Run BFS from b to e in the expanded graph; the answer is $\text{dist}(b, e)/2$. The expanded graph has $O(N + M)$ vertices and edges.

3.2 Game on Axis

Link: https://codeforces.com/problemset/problem/1787/D
Prerequisites: Functional directed graphs, cycle detection, reversed sink-tree subtree sizes.

There are n points $1, 2, \ldots, n$ ($\sum n \leq 2 \cdot 10^5$) on a number line, each with an integer jump offset a_i ($-n \leq a_i \leq n$). A token starts at point 1; whenever it is at $x \in [1, n]$, it jumps to $x + a_x$. The game terminates if the token jumps outside $[1, n]$, and runs forever if it enters a cycle. Before the game starts, you choose one index $i \in \{1, \ldots, n\}$ and replace a_i with an integer $y \in [-n, n]$.

Task: Count how many of the $n(2n + 1)$ possible pairs (i, y) result in a game that terminates in a finite number of steps.

Hint: Functional Graphs

Treat out-of-bounds jumps as edges to a terminal sink 0. In the reversed graph, the vertices that terminate form a tree rooted at 0.

Full worked proofs, complexity analyses, and implementations in Python and C++ are provided in ***Week 4 Solutions.pdf****.*